🌐 CIDR & IPv4 — Complete Interview Notes (AWS Networking Foundation)

> 🧠 If you master CIDR → You master VPC design → You master AWS networking.

This is **EXTREMELY important** for interviews.
Almost every AWS networking question depends on CIDR understanding.

📌 What is CIDR?

CIDR = Classless Inter-Domain Routing

It is a method for:

- Allocating IP addresses
- Defining IP ranges
- Designing subnets

In AWS, you constantly see:

```
10.0.0.0/16
192.168.1.0/24
0.0.0.0/0
```

That `/number` is the CIDR notation.

🧩 CIDR Structure

A CIDR block has **2 components**:

```
<Base IP> / <Subnet Mask Length>
```

Example:

```
192.168.0.0/24
```

Component	Meaning
192.168.0.0	Base IP
/24	Subnet mask length

🔢 What Does “/Number” Mean?

The `/number` tells us:

> How many bits are FIXED for the network.

IPv4 address = 32 bits total

So:

```
Number of usable IPs = 2^(32 - subnet mask)
```

🎯 Quick Memory Rule (VERY IMPORTANT)

An IPv4 address has **4 octets**:

```
192.168.0.0
 |    |   |   |
 1st  2nd 3rd 4th
```

Now remember this:

CIDR	What Changes?
/32	Nothing changes
/24	Last octet changes
/16	Last 2 octets change
/8	Last 3 octets change
/0	Everything changes

📊 CIDR Size Table (Interview Favorite)

CIDR	Total IPs	Formula
/32	1	2^{0}
/31	2	2^{1}
/30	4	2^{2}
/29	8	2^{3}
/28	16	2^{4}
/27	32	2^{5}
/26	64	2^{6}
/25	128	2^{7}
/24	256	2^{8}
/16	65,536	2^{16}

CIDR	Total IPs	Formula
/8	16,777,216	2^{24}
/0	All IPv4	2^{32}

🔍 Examples Explained Clearly

✅ 192.168.0.0/24

- /24 → last octet changes
- Range:

```
192.168.0.0 → 192.168.0.255
```

- Total IPs = 256

✅ 192.168.0.0/26

- /26 → 32 - 26 = 6 bits free
- 2^6 = 64 IPs

Range:

```
192.168.0.0 → 192.168.0.63
```

✅ 192.168.0.0/16

- Last 2 octets change
- 65,536 IPs

Range:

```
192.168.0.0 → 192.168.255.255
```

✅ 134.56.78.123/32

- Only ONE IP
- Used often in security groups to allow a specific machine

✅ 0.0.0.0/0

🔥 VERY IMPORTANT

Means:

> ALL IPv4 addresses

Used in:

- Security groups (allow internet)
- Route tables (default route)

🧠 How CIDR Works Internally (Bit Level)

Example: `/24`

```
11111111.11111111.11111111.00000000
```

First 24 bits fixed → last 8 bits variable.

Binary = Decimal:

```
00000000 → 0
11111111 → 255
```

So:

```
192.168.0.0 → 192.168.0.255
```

🌍 Public vs Private IPs

This is defined by the
Internet Assigned Numbers Authority (IANA)

They reserved special private ranges.

🔒 Private IP Ranges (VERY IMPORTANT)

Range	CIDR	Usage
10.0.0.0 – 10.255.255.255	10.0.0.0/8	Large networks

Range	CIDR	Usage
172.16.0.0 – 172.31.255.255	172.16.0.0/12	Medium networks
192.168.0.0 – 192.168.255.255	192.168.0.0/16	Home routers

🏠 Home Network Example

Most home WiFi routers use:

```
192.168.1.x
```

That comes from:

```
192.168.0.0/16
```

☁️ How This Connects to AWS VPC

When you create a VPC, you choose a CIDR.

Example:

```
10.0.0.0/16
```

Then you create subnets:

```
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
```

ASCII visualization:

```
VPC (10.0.0.0/16)
│
├── Subnet A (10.0.1.0/24)
├── Subnet B (10.0.2.0/24)
└── Subnet C (10.0.3.0/24)
```

⚠️ AWS Special Rule (Interview Trap)

AWS reserves **5 IP addresses per subnet**.

For example:

In `/24` (256 IPs):

You get:

```
256 - 5 = 251 usable IPs
```

Reserved:

- Network address
- Router
- DNS
- Future use
- Broadcast (not technically used in AWS but reserved)

🔦 Interview Questions

1️⃣ What does /24 mean?

It means:

- 24 bits fixed
- 8 bits free
- 256 IPs total

2️⃣ What does 0.0.0.0/0 mean?

All IPv4 addresses.

3️⃣ How many IPs in /26?

2^(32-26) = 2^6 = 64 IPs.

4️⃣ What CIDR is used for home networks?

192.168.0.0/16.

5️⃣ What are private IP ranges?

- 10.0.0.0/8

- 172.16.0.0/12
- 192.168.0.0/16

6️⃣ Why use private IPs?

They:

- Are not routable on internet
- Increase security
- Prevent IP exhaustion

🚀 Real-World AWS Scenario

Suppose:

You create:

```
VPC → 10.0.0.0/16
```

You want:

- 2 public subnets
- 2 private subnets

You could design:

```
Public-1   → 10.0.1.0/24
Public-2   → 10.0.2.0/24
Private-1  → 10.0.3.0/24
Private-2  → 10.0.4.0/24
```

Each gives:

```
256 IPs (251 usable in AWS)
```

🔥 Pro Tips for Interviews

✅ Always know powers of 2
✅ Know /24 = 256
✅ Know /16 = 65,536
✅ Know private ranges by heart

✅ Understand subnet splitting
✅ Understand AWS reserves 5 IPs

🧠 Mental Shortcut Trick

Remember this ladder:

```
/32 → 1 IP
/31 → 2
/30 → 4
/29 → 8
/28 → 16
/27 → 32
/26 → 64
/25 → 128
/24 → 256
```

Just double as you go up ⬆️

🎯 Final Takeaways

✅ CIDR defines IP ranges
✅ /number defines network size
✅ Smaller number = larger network
✅ Private IP ranges are predefined
✅ AWS VPC requires CIDR knowledge
✅ AWS reserves 5 IPs per subnet

🌐 Default VPC in AWS — Complete Interview Notes

> 🧠 If you understand the **default VPC**, you understand why your EC2 worked "magically" without networking setup.

AWS creates a **default VPC automatically** so beginners can launch resources immediately.

🎯 Why Does AWS Create a Default VPC?

Without a VPC:

- ❌ You cannot launch EC2
- ❌ No networking
- ❌ No internet access

So AWS gives you:

```
Default VPC (Ready-to-Use Network)
```

This allows you to:

- Launch EC2 immediately
- Get internet access instantly
- Avoid manual networking setup

🏗️ Default VPC Architecture (High-Level)

```
                🌍 Internet
                     |
               🚪 Internet Gateway
                     |
          -------------------------
          |           |           |
       Subnet A    Subnet B    Subnet C
        (AZ-1)      (AZ-2)      (AZ-3)
          |           |           |
         EC2         EC2         EC2
```

✅ 1 VPC
✅ 3 Subnets
✅ 3 Availability Zones
✅ Internet Gateway attached
✅ Public IP auto-enabled

📦 What Comes With a Default VPC?

When you create a new AWS account, AWS automatically creates:

Component	Created by Default?
VPC	✅ Yes
Subnets (3)	✅ Yes

Component	Created by Default?
Internet Gateway	✅ Yes
Route Table	✅ Yes
Network ACL	✅ Yes
Security Group	✅ Yes

🧩 Default VPC Details

1️⃣ VPC CIDR Block

Typically:

```
172.31.0.0/16
```

That means:

- 65,536 total IPs
- Last 2 octets can change

```
172.31.0.0 → 172.31.255.255
```

2️⃣ Subnets

AWS creates **3 subnets**, each in a different AZ.

Example:

```
Subnet 1 → 172.31.0.0/20
Subnet 2 → 172.31.16.0/20
Subnet 3 → 172.31.32.0/20
```

Each `/20`:

- 2^(32-20) = 4096 IPs
- BUT AWS reserves 5 IPs
- Usable = 4091

⚠️ Interview Trap:

> Why 4091 and not 4096?

Because AWS reserves 5 IPs per subnet.

🔐 Why EC2 Gets Public IP Automatically?

Because:

✅ Subnet setting → **Auto-assign public IPv4 = ENABLED**
✅ Route table has route to Internet Gateway
✅ IGW is attached to VPC

So traffic flows like this:

```
EC2 (Public IP)
     |
     v
Route Table (0.0.0.0/0 → IGW)
     |
     v
Internet Gateway
     |
     v
▫ Internet
```

That's why SSH worked immediately.

🗺️ Route Table in Default VPC

Main Route Table contains:

Destination	Target
172.31.0.0/16	local
0.0.0.0/0	Internet Gateway

🔍 What Does "local" Mean?

Allows internal communication inside VPC.

Example:

```
EC2-A → EC2-B (Same VPC)
```

Traffic never leaves AWS.

🔥 What Does 0.0.0.0/0 Mean?

All IPv4 traffic goes to:

```
Internet Gateway
```

That gives internet access.

🌐 Internet Gateway (IGW)

Attached to default VPC.

Purpose:

- Allows inbound & outbound internet traffic

Without IGW:

- No public access
- No internet connectivity

🔒 Default Network ACL

Default behavior:

Direction	Rule
Inbound	Allow all
Outbound	Allow all

That means:

- Everything is allowed
- No restrictions

⚠️ Production environments should NOT use this as-is.

🧠 Subnet & Route Table Association

In default VPC:

- Subnets are NOT explicitly associated
- They use the **Main Route Table implicitly**

Important concept:

```
If subnet has no custom route table →
It uses Main Route Table
```

🚨 Why Not Use Default VPC in Production?

Best Practice:

> ❌ Do NOT use default VPC for production workloads.

Why?

- Too permissive
- Hard to control
- No separation of public/private
- Less secure

Instead:

```
Create Custom VPC
    ├── Public Subnets
    ├── Private Subnets
    ├── NAT Gateway
    └── Controlled Route Tables
```

🎤 Interview Questions

1️⃣ What is a Default VPC?

A VPC automatically created by AWS in each region with:

- Subnets
- IGW
- Route table
- Public IP auto-enabled

2️⃣ How many subnets in default VPC?

Usually 3 (one per AZ).

3️⃣ Why do EC2 instances in default VPC get internet access?

Because:

- Public IP assigned
- Route table has 0.0.0.0/0 → IGW
- IGW attached

4️⃣ Why are there 4091 IPs available instead of 4096?

AWS reserves 5 IPs per subnet.

5️⃣ What happens if a subnet has no route table explicitly attached?

It uses the main route table implicitly.

🔥 Real Interview Scenario

Interviewer:

> You launched EC2 in default VPC without configuring networking. Why did it work?

Strong Answer:

Because the default VPC:

- Has an attached Internet Gateway
- Has a route table with 0.0.0.0/0 route
- Has subnets with auto-assign public IP enabled
- Has permissive NACL

Therefore, EC2 gets public IP and internet access automatically.

🧠 Mental Model

Think of Default VPC as:

```
"Beginner Mode Networking"
```

Everything is:

- Open
- Simple
- Internet-enabled

But not production-ready.

🏁 Final Takeaways

✅ Default VPC is auto-created
✅ Comes with 3 public subnets
✅ Internet Gateway attached
✅ Route table allows internet
✅ Public IP auto-enabled
✅ AWS reserves 5 IPs per subnet
❌ Not recommended for production

🌐 Creating Your First VPC in AWS — Interview Notes

> 🧠 VPC = **Virtual Private Cloud**
> Your isolated private network inside AWS.

Now we move from **default VPC (auto-created)** to creating a **custom VPC** — this is what real-world architecture uses.

🏗️ What is a VPC?

A VPC is:

```
Your Own Private Network in AWS
```

Inside it, you launch:

- EC2
- RDS
- Load Balancers
- Lambda (VPC-enabled)
- ECS/EKS

Think of it as:

```
AWS Region
    ├── VPC 1
    ├── VPC 2
```

```
└── VPC 3
```

📌 Important Limits (Interview Critical)

🧮 Number of VPCs per Region

- Default: **5 VPCs per region**
- ⚠️ Soft limit → can request increase

🧮 CIDR Blocks per VPC

- Maximum: **5 CIDRs per VPC**

This means:

```
1 primary CIDR
+ 4 secondary CIDRs (optional)
```

Used when:

- Expanding IP space
- Avoiding redesign

📏 VPC CIDR Size Rules

When creating a VPC:

Rule	Value
Minimum size	/28
Maximum size	/16

🔢 What Does That Mean?

/28

```
2^(32-28) = 16 IP addresses
```

Minimum allowed.

/16

```
2^(32-16) = 65,536 IP addresses
```

Maximum allowed.

🔐 Only Private IPv4 Allowed

Since VPC is private, you must use private IP ranges.

Reserved by
Internet Assigned Numbers Authority

🔒 Private IP Ranges

Range	CIDR
10.0.0.0 – 10.255.255.255	10.0.0.0/8
172.16.0.0 – 172.31.255.255	172.16.0.0/12
192.168.0.0 – 192.168.255.255	192.168.0.0/16

You must choose CIDR from these ranges.

🚨 VERY IMPORTANT — CIDR Overlapping

When choosing your VPC CIDR:

> ❗ It must NOT overlap with:

- Another VPC
- On-prem corporate network
- Peered VPC
- VPN-connected network

🔥 Why Is Overlapping Bad?

Suppose:

```
VPC A → 10.0.0.0/16
```

```
VPC B → 10.0.0.0/16
```

Now you try VPC Peering ❌

Routing confusion:

```
Where does 10.0.1.5 belong?
```

AWS cannot differentiate.

So always design carefully.

🧠 Real-World Design Strategy

If company uses:

```
10.0.0.0/8 internally
```

Then in AWS:

- Use 172.16.0.0/12
- Or 192.168.0.0/16

Avoid overlap at all costs.

🏗️ Example: Creating a Simple Custom VPC

Let's say we create:

```
VPC CIDR: 10.0.0.0/16
```

This gives:

- 65,536 total IPs

Now later we divide:

```
10.0.1.0/24 → Public Subnet
10.0.2.0/24 → Public Subnet
10.0.3.0/24 → Private Subnet
10.0.4.0/24 → Private Subnet
```

📊 CIDR Planning Strategy (Interview Gold)

Step 1: Choose VPC Size

Small project?

```
/20 or /21
```

Medium:

```
/18 or /17
```

Large production:

```
/16
```

Step 2: Leave Room for Growth

Don't use:

`10.0.0.0/24`

Too small for scaling.

Better:

`10.0.0.0/16`

Then subnet it properly.

🎤 Interview Questions

1️⃣ How many VPCs per region?

Default 5 (soft limit).

2️⃣ What is minimum CIDR size for VPC?

/28 (16 IPs).

3️⃣ What is maximum CIDR size?

/16 (65,536 IPs).

4️⃣ Can VPC use public IP ranges?

No ❌
Only private IPv4 ranges allowed.

5️⃣ Why must VPC CIDR not overlap?

Because:

- Routing conflicts
- Peering fails
- VPN conflicts
- Transit Gateway issues

6️⃣ Can you expand a VPC later?

Yes ✅
By adding up to 4 secondary CIDRs (total 5).

🧠 Architecture Visualization

```
AWS Region
   |
   └── VPC (10.0.0.0/16)
           |
           ├── Public Subnet
           ├── Private Subnet
           ├── Route Tables
           ├── Internet Gateway
           └── NAT Gateway
```

🚀 Pro Tips for Interviews

✅ Always mention CIDR overlap prevention
✅ Mention soft limits vs hard limits
✅ Mention 5 CIDRs per VPC
✅ Mention /28 minimum
✅ Mention /16 maximum
✅ Talk about future scalability

🎯 Final Takeaways

✔ You can have multiple VPCs per region
✔ 5 VPCs default limit (soft)
✔ 5 CIDRs per VPC
✔ CIDR must be between /28 and /16
✔ Only private IP ranges allowed
✔ Never overlap CIDRs

🏗️ Creating a Custom VPC Manually (No Wizard) — Interview Notes

> 🧠 Real engineers don't rely on the VPC Wizard.
> To truly understand networking, you must create everything step-by-step.

🎯 Why NOT Use the VPC Wizard?

The wizard:

- Creates everything automatically
- Hides networking details
- Prevents deep understanding

Manual setup helps you understand:

- CIDR planning
- Route tables
- NACLs
- Internet connectivity
- Tenancy

🧱 Step 1: Create the VPC

Example Configuration

Setting	Value
Name	DemoVPC
IPv4 CIDR	10.0.0.0/16
IPv6	None
Tenancy	Default

🌐 CIDR Selection Explained

We chose:

```
10.0.0.0/16
```

Why?

- It’s within private IP range
- Large enough (65,536 IPs)
- Good for scaling

🔢 CIDR Size Check

```
/16 → 2^(32-16) = 65,536 IPs
```

Range:

```
10.0.0.0 → 10.0.255.255
```

⚠️ If you try:

```
10.0.0.0/15
```

It fails ❌

Why?

Because AWS VPC maximum CIDR size is:

```
/16
```

🔐 IPv6 Option

During creation:

- You can assign IPv6 CIDR
- We skipped it for simplicity

Important:

- IPv6 support can be added later
- Works differently from IPv4

🏢 Tenancy Explained (Interview Important)

Two options:

Tenancy	Meaning
Default	Shared hardware
Dedicated	Dedicated hardware

Default Tenancy

- EC2 runs on shared AWS infrastructure
- Most common
- Cheapest option
- Used in 99% of workloads

🔴 Dedicated Tenancy

- Physical hardware dedicated to you
- Required for:
  - Compliance
  - Licensing restrictions
  - Certain enterprise use cases

⚠️ Very expensive.

🏗️ What AWS Creates Automatically?

When you create a VPC manually, AWS automatically creates:

Component	Created?
Main Route Table	✅ Yes
Main Network ACL	✅ Yes
Subnets	❌ No
Internet Gateway	❌ No

🗺️ Main Route Table (Auto-Created)

Every VPC must have:

```
1 Main Route Table
```

Initially contains:

```
Destination: 10.0.0.0/16
```

```
Target: local
```

This allows internal communication inside VPC.

🔐 Main Network ACL (Auto-Created)

Default NACL:

- Allows all inbound
- Allows all outbound

You can customize later.

🔄 Adding Additional CIDRs to VPC

After VPC creation:

You can:

`Actions → Edit CIDRs → Add CIDR`

Example:

`10.1.0.0/16`

Now VPC has:

```
Primary CIDR: 10.0.0.0/16
Secondary CIDR: 10.1.0.0/16
```

📌 CIDR Rules Recap

- Max 5 IPv4 CIDRs per VPC
- Each must be between /28 and /16
- Must not overlap

🧠 Why Add Secondary CIDR?

Used when:

- You run out of IPs
- Expanding infrastructure
- Merging networks
- Large microservices architecture

Example:

```
VPC
 ├── 10.0.0.0/16
 └── 10.1.0.0/16
```

Then create subnets from both ranges.

🏗️ Architecture So Far

Right now we have:

```
AWS Region
   |
   └── DemoVPC (10.0.0.0/16)
          |
          ├── Main Route Table
          └── Main Network ACL
```

🚫 No subnets yet
🚫 No internet gateway
🚫 No EC2
🚫 No internet access

🎤 Interview Questions

1️⃣ What happens when you create a VPC?

AWS automatically creates:

- Main route table
- Main NACL

2️⃣ What is VPC Tenancy?

Defines whether EC2 runs on:

- Shared hardware (Default)
- Dedicated hardware

3️⃣ What is maximum CIDR size allowed for VPC?

/16

4️⃣ Can you add more CIDRs later?

Yes. Up to 5 total IPv4 CIDRs.

5️⃣ Why would you add secondary CIDRs?

To expand IP space without recreating VPC.

🚀 Common Beginner Mistakes

❌ Choosing too small CIDR (/24 VPC)
❌ Not planning future growth
❌ Overlapping corporate network
❌ Using Dedicated tenancy unnecessarily
❌ Thinking subnets are auto-created (they are NOT)

🧠 Mental Model

Creating a VPC manually is like:

```
Buying Empty Land
```

Right now:

- No roads
- No buildings
- No gates
- Just land with boundaries

Next steps will be:

- Create subnets (divide land)
- Add internet gateway (main gate)
- Create route tables (roads)
- Add NAT (controlled exit)

🎯 Final Takeaways

✅ VPC is created with CIDR
✅ Max CIDR size = /16
✅ Min CIDR size = /28
✅ Main route table auto-created
✅ Main NACL auto-created
✅ Can add up to 5 CIDRs
✅ Default tenancy = shared hardware

🔥 You just created the foundation of real AWS networking.

Next step is usually creating subnets — where things get interesting 🚀

Send the next transcript and we continue building your VPC from scratch 💪

🧱 Subnets in AWS VPC — Complete Interview Notes

> 🧠 A subnet is a **smaller network inside your VPC**.
> It divides your VPC into logical sections (public/private).

Now we're creating:

- 🌐 1 Public Subnet
- 🔒 1 Private Subnet
- Both inside the same Availability Zone

📌 What is a Subnet?

A **subnet** is:

```
A sub-range of IP addresses from your VPC CIDR
```

Example:

```
VPC → 10.0.0.0/16
```

```
Subnet → 10.0.1.0/24
```

So subnet is just a portion of the VPC IP space.

🧠 Visual Understanding

```
VPC (10.0.0.0/16)
│
├── Public Subnet (10.0.1.0/24)
└── Private Subnet (10.0.2.0/24)
```

Each subnet:

- Lives in ONE Availability Zone
- Has its own CIDR block
- Uses IPs from the VPC range

🚨 AWS Reserves 5 IPs Per Subnet (Very Important)

In every subnet, AWS reserves:

IP	Purpose
x.x.x.0	Network address
x.x.x.1	VPC Router
x.x.x.2	Amazon DNS
x.x.x.3	Reserved (future use)
x.x.x.255 (last IP)	Broadcast (reserved)

⚠️ Even though AWS does NOT support broadcast, that IP is still reserved.

🔍 Example: 10.0.0.0/24

Total IPs:

```
2^(32-24) = 256 IPs
```

Range:

```
10.0.0.0 → 10.0.0.255
```

Reserved:

```
10.0.0.0   → Network
10.0.0.1   → Router
10.0.0.2   → DNS
10.0.0.3   → Reserved
```

`10.0.0.255 → Broadcast`

Usable:

`256 - 5 = 251 IPs`

🎯 Why This Matters in Interviews

Scenario:

Interviewer says:

> You need 29 EC2 instances in a subnet. What CIDR do you choose?

Option 1: /27

`2^(32-27) = 32 IPs`

`32 - 5 reserved = 27 usable`

❌ Not enough (need 29)

Option 2: /26

`2^(32-26) = 64 IPs`

`64 - 5 = 59 usable`

✅ Correct choice

📊 Quick Subnet Size Table (After AWS Reservation)

CIDR	Total IPs	Usable IPs
/28	16	11
/27	32	27
/26	64	59
/25	128	123
/24	256	251

🌐 Public vs Private Subnet (Important Concept)

A subnet is NOT automatically public or private.

It depends on:

```
Route Table + Internet Gateway
```

🌐 Public Subnet

A subnet is public if:

```
Route Table → 0.0.0.0/0 → Internet Gateway
```

AND

```
Auto-assign Public IP enabled
```

🔒 Private Subnet

A subnet is private if:

```
No direct route to Internet Gateway
```

Usually:

```
0.0.0.0/0 → NAT Gateway
```

🧠 Architecture So Far

After creating VPC:

```
DemoVPC (10.0.0.0/16)
    |
    ├── Public Subnet (10.0.1.0/24)
    └── Private Subnet (10.0.2.0/24)
```

Currently:

- No Internet Gateway yet
- No custom route tables yet
- Subnets not yet public/private logically

🎤 Interview Questions

1️⃣ How many IPs are reserved per subnet?

5 IP addresses.

2️⃣ Can you use the first IP in a subnet?

No. Reserved for network.

3️⃣ What is x.x.x.1 used for?

VPC router.

4️⃣ What is x.x.x.2 used for?

Amazon-provided DNS.

5️⃣ Does AWS support broadcast?

No. But the broadcast IP is still reserved.

6️⃣ If you need 100 EC2 instances, what subnet size?

Need at least 100 usable.

Try:

`/25 → 128 total → 123 usable`

✅ Good choice

🚀 Common Beginner Mistakes

❌ Forgetting AWS reserves 5 IPs
❌ Choosing subnet too small
❌ Thinking subnet is automatically public
❌ Creating overlapping subnet CIDRs
❌ Forgetting subnet must belong to ONE AZ

🧠 Mental Model

Think of:

```
VPC = Big Land
Subnet = Neighborhood
Reserved IPs = Utility infrastructure
```

You cannot build houses on:

- Roads
- Electrical grid
- DNS system
- Broadcast infrastructure

🎯 Final Takeaways

✅ Subnet = portion of VPC CIDR
✅ Each subnet belongs to ONE AZ
✅ AWS reserves 5 IPs per subnet
✅ Public/private depends on routing
✅ Always calculate usable IPs

🧱 Creating Subnets in Your Custom VPC — Interview Notes

> 🧠 Subnets divide your VPC into smaller, organized networks across Availability Zones (AZs).

We created:

- 🌐 2 Public Subnets
- 🔒 2 Private Subnets
- Across 2 Availability Zones
- Inside `DemoVPC (10.0.0.0/16)`

🎯 Step 1: Filter by VPC (Best Practice)

When creating subnets:

```
VPC Console → Subnets → Filter by DemoVPC
```

Why?

✅ Avoid confusion
✅ Prevent creating subnets in wrong VPC
✅ Cleaner UI

🏗️ Subnet Architecture We Built

```
DemoVPC (10.0.0.0/16)
│
├── 🌐 PublicSubnetA  (10.0.0.0/24)  → eu-central-1a
├── 🌐 PublicSubnetB  (10.0.1.0/24)  → eu-central-1b
│
├── 🔒 PrivateSubnetA (10.0.16.0/20) → eu-central-1a
└── 🔒 PrivateSubnetB (10.0.32.0/20) → eu-central-1b
```

📌 Why Two AZs?

We used:

- eu-central-1a
- eu-central-1b

This gives:

✅ High Availability
✅ Fault Tolerance
✅ Multi-AZ Architecture

If one AZ fails → the other still works.

🌐 Public Subnets — CIDR Choice

PublicSubnetA

```
10.0.0.0/24
```

PublicSubnetB

```
10.0.1.0/24
```

Why /24 for Public?

```
/24 → 256 total IPs
```

```
256 - 5 reserved = 251 usable
```

Public subnets usually host:

- Load Balancers
- Bastion Hosts
- NAT Gateway
- Reverse Proxies

They typically don’t need thousands of IPs.

So /24 is practical and cost-efficient.

🔒 Private Subnets — CIDR Choice

PrivateSubnetA

```
10.0.16.0/20
```

PrivateSubnetB

```
10.0.32.0/20
```

Why /20 for Private?

```
/20 → 2^(32-20) = 4096 total IPs
```

```
4096 - 5 = 4091 usable
```

Private subnets host:

- Application servers
- Databases
- Internal microservices
- Auto Scaling groups

They usually need more IP space.

🧠 Why 10.0.16.0/20 After 10.0.1.0/24?

We must avoid overlapping CIDRs.

We had:

`Public A → 10.0.0.0/24`

`Public B → 10.0.1.0/24`

Next logical larger block:

`10.0.16.0/20`

Range of /20:

`10.0.16.0 → 10.0.31.255`

Then next block:

`10.0.32.0/20`

Range:

`10.0.32.0 → 10.0.47.255`

Everything is clean and non-overlapping ✅

🚨 AWS Reserved IP Reminder

Each subnet loses 5 IPs.

So:

CIDR	Total	Usable
/24	256	251
/20	4096	4091

Interview Tip:

> Always subtract 5 in AWS.

⚠️ Important: Are They Public Yet?

Answer:

❌ NO.

Right now, all subnets are identical.

A subnet becomes **public** only if:

`Route Table → 0.0.0.0/0 → Internet Gateway`

A subnet becomes **private** if:

```
No direct route to Internet Gateway
```

We have not configured route tables yet.

🧠 Big Picture So Far

```
Internet ❌ (Not connected yet)


DemoVPC
│
├── PublicSubnetA (No IGW route yet)
├── PublicSubnetB (No IGW route yet)
├── PrivateSubnetA
└── PrivateSubnetB
```

Next steps will likely be:

- Attach Internet Gateway
- Create Public Route Table
- Associate public subnets
- Create NAT Gateway
- Create Private Route Table

🎤 Interview Questions

1️⃣ Can a subnet span multiple AZs?

No ❌
A subnet exists in exactly ONE AZ.

2️⃣ How do you achieve high availability?

Create subnets in multiple AZs.

3️⃣ Why make public subnets smaller?

They host fewer resources (load balancers, NAT, bastion).

4️⃣ How do you avoid overlapping CIDRs?

Plan ranges carefully and calculate boundaries.

5️⃣ Are subnets automatically public?

No. Public/private is defined by routing.

🚀 Common Mistakes

❌ Overlapping subnet CIDRs
❌ Creating subnet in wrong AZ
❌ Making all subnets /24 without planning growth
❌ Forgetting AWS reserves 5 IPs
❌ Thinking naming makes it public (it does NOT)

🧠 Mental Model

Think of:

```
VPC = Big City
```

```
Subnets = Districts
```

```
AZ = Different Physical Locations
```

You created districts in two physical cities.

But you haven't built highways (route tables) yet.

🎯 Final Takeaways

✅ Subnets divide VPC IP space
✅ Each subnet belongs to one AZ
✅ Use multiple AZs for HA
✅ Public subnets often smaller
✅ Private subnets often larger
✅ AWS reserves 5 IPs per subnet
✅ Routing defines public/private

🌍 Internet Gateway (IGW) — Enabling Internet Access in Your VPC

🧠 Right now, your subnets are just isolated networks.
They have **NO internet access** until we attach an Internet Gateway and update routing.

🎯 What is an Internet Gateway?

An **Internet Gateway (IGW)** is a managed AWS component that:

✅ Allows VPC resources to access the Internet
✅ Enables inbound & outbound internet traffic
✅ Is highly available and horizontally scalable
✅ Is fully managed by AWS

🏗️ Key Rules About Internet Gateway (Interview Important)

Rule	Explanation
1 IGW per VPC	A VPC can attach only ONE IGW
1 VPC per IGW	IGW cannot attach to multiple VPCs
Must be attached manually	Created separately from VPC
IGW alone does NOT give internet	Route table must be updated

🧠 Important Concept

Creating an Internet Gateway is NOT enough.

You must:

1️⃣ `Create IGW`

2️⃣ `Attach IGW to VPC`

3️⃣ `Update Route Table`

4️⃣ `Associate Route Table with Public Subnet`

Only then does it become a **Public Subnet**.

🌐 Current Architecture (Before IGW)

```
DemoVPC
│
├── Public Subnet A ❌ (No Internet)
├── Public Subnet B ❌
├── Private Subnet A ❌
└── Private Subnet B ❌
```

All subnets are currently PRIVATE.

🏗️ After Creating Internet Gateway (Still No Internet)

```
                🌍 Internet
                     |
                🚪 Internet Gateway
                     |
                 DemoVPC
                     |
        -------------------------
        |                       |
   Public Subnet A          Private Subnet A
```

Still ❌ No internet access.

Why?

Because traffic doesn't know where to go.

🗺️ Role of Route Tables

To enable internet access:

Add this route:

```
Destination: 0.0.0.0/0
Target: Internet Gateway
```

This means:

Any traffic going outside VPC → send to IGW.

🔥 Final Working Flow

```
EC2 (Public Subnet)
       |
       v
Route Table (0.0.0.0/0 → IGW)
       |
       v
Internet Gateway
       |
       v
▫ Internet
```

Now the subnet becomes PUBLIC.

📌 What Makes a Subnet Public?

A subnet is PUBLIC if:

✅ It has a route to IGW (0.0.0.0/0 → IGW)
✅ Instances have public IP enabled

If either is missing → No internet.

🔒 What About Private Subnets?

Private subnets:

❌ Do NOT have route to IGW
✅ May later use NAT Gateway

🎤 Interview Questions

1️⃣ Does attaching an Internet Gateway automatically make subnets public?

No ❌
You must update route tables.

2️⃣ Can a VPC have multiple Internet Gateways?

No. Only one per VPC.

3️⃣ Is Internet Gateway highly available?

Yes. It is managed, scalable, and redundant.

4️⃣ What route enables internet access?

`0.0.0.0/0 → Internet Gateway`

5️⃣ What does 0.0.0.0/0 mean?

All IPv4 traffic.

🚨 Common Mistakes

❌ Creating IGW but not attaching it
❌ Attaching IGW but not updating route table
❌ Forgetting to enable public IP on EC2
❌ Associating wrong route table
❌ Thinking subnet name makes it public

🧠 Mental Model

Think of:

```
VPC = House

Internet Gateway = Main Gate

Route Table = Road Map
```

If there is no road pointing to the gate,
you cannot exit the house.

🎯 Final Takeaways

✅ IGW enables internet access
✅ Must attach IGW to VPC
✅ Must update route table
✅ Subnet becomes public only after routing
✅ IGW is highly available and scalable

🌐 Enabling Internet Access in DemoVPC — Full Walkthrough (Interview Notes)

Now we'll validate something important:

> ❗ Even if an EC2 has a Public IP, it does NOT automatically mean it has internet access.

Let's break this step-by-step and expand everything for interview clarity 👇

🧪 Step 1 — Launch EC2 in Public Subnet (Test Before IGW Routing)

You launched:

- AMI: Amazon Linux 2
- Instance Type: t2.micro
- VPC: DemoVPC
- Subnet: PublicSubnetA
- Security Group: SSH (Port 22 open)
- Public IP: Enabled

Instance was created and got:

✅ `Public IPv4 address`

✅ `Private IPv4 address`

But…

❌ EC2 Instance Connect FAILED
❌ Could not SSH
❌ No internet connectivity

🧠 Why Did It Fail?

Because:

`Public IP ≠ Internet Access`

Internet access requires **3 things working together**:

1️⃣ `Public IP`

2️⃣ `Internet Gateway (IGW) attached`

3️⃣ `Route Table entry → 0.0.0.0/0 → IGW`

You only had #1.

🌍 Step 2 — Create Internet Gateway

You created:

DemoIGW

Then attached it to:

`DemoVPC`

Architecture now:

```
🌍 Internet
     |
🚪 DemoIGW
     |
  DemoVPC
     |
PublicSubnetA
     |
   EC2
```

But still…

❌ No internet access

🧠 Why Still Not Working?

Because traffic still doesn't know where to go.

An IGW alone does NOTHING until routing is configured.

🗺️ Step 3 — Fix Route Tables (Critical Step 🔥)

Originally:

All subnets were using the **Main Route Table**.

That works, but it's bad practice.

Interview Tip 💡
Always create separate route tables for:

- Public subnets
- Private subnets

🏗️ You Created:

PublicRouteTable

Associated with:

- PublicSubnetA
- PublicSubnetB

🔵 PrivateRouteTable

Associated with:

- PrivateSubnetA
- PrivateSubnetB

Now routing becomes explicit and clean.

📌 Default Route Explained

Inside route table you saw:

```
Destination: 10.0.0.0/16
```

```
Target: local
```

What does this mean?

It means:

```
All traffic inside VPC → stay local
```

This route is automatically created and cannot be removed.

🌐 Step 4 — Add Internet Route

In PublicRouteTable you added:

```
Destination: 0.0.0.0/0
Target: DemoIGW
```

Meaning:

```
Any traffic NOT inside VPC → send to Internet Gateway
```

Now full traffic flow looks like this:

```
EC2 (PublicSubnetA)
        |
        v
PublicRouteTable
        |
        | 0.0.0.0/0
        v
DemoIGW
        |
        v
▫ Internet
```

Now it works ✅

🧪 Verification

You connected using EC2 Instance Connect.

Then tested:

```
ping google.com
```

You received responses 🎉

That confirms:

✅ Outbound internet working
✅ Routing correct
✅ IGW attached
✅ Public IP functioning

🧠 What Makes a Subnet Public?

A subnet becomes PUBLIC if:

✅ `Has route 0.0.0.0/0 → IGW`

✅ `Instances have public IP`

If either is missing → it's PRIVATE.

🔒 What About Private Subnets?

Your private subnets:

- Have NO route to IGW
- Only have local route

So they cannot access internet.

Later you'll use:

`NAT Gateway`

To give outbound-only internet access.

🎯 Interview Deep Understanding

❓ Why Did EC2 Fail Even With Public IP?

Because route table had no default route to IGW.

❓ Can you SSH if:

- Public IP enabled ✅
- Security Group allows SSH ✅
- IGW attached ✅
- Route missing ❌

Answer: No.

Routing is mandatory.

❓ What does 0.0.0.0/0 mean?

It represents:

```
All IPv4 addresses
```

It is called the **Default Route**.

❓ Why create separate route tables?

Best practice:

```
Public Subnets → Route to IGW
```

```
Private Subnets → No IGW route
```

Better security and control.

⚠️ Common Interview Traps

❌ Thinking IGW alone gives internet
❌ Thinking public IP is enough
❌ Forgetting route table association
❌ Using Main Route Table carelessly
❌ Forgetting Security Group rules

🏗️ Full Architecture Now

```
                    🌍 Internet
                         |
                    🚪 DemoIGW
                         |
                 -----------------
                 |               |
        PublicRouteTable     PrivateRouteTable
         (0.0.0.0/0 → IGW)        (No IGW)
                 |               |
       -------------------    -------------------
       |                 |    |                 |
PublicSubnetA      PublicSubnetB    PrivateSubnetA    PrivateSubnetB
       |                                       |
```

```
EC2 (Internet)                    EC2 (No Internet)
```

🧠 Big Picture Concept

Think of it like this:

```
Internet Gateway = Exit Door
Route Table = Road Map
Public IP = Your Address
Security Group = Security Guard
```

All must cooperate.

🚀 Final Takeaways

✅ Public IP alone is NOT enough
✅ IGW must be attached
✅ Route table must have 0.0.0.0/0 → IGW
✅ Subnet association is critical
✅ Best practice = separate public/private route tables

🛡️ Bastion Host — Accessing Private EC2 Instances (Interview Notes)

When dealing with private subnets, **direct internet access is impossible**. Bastion hosts are the solution.

🔑 Concept

- **Problem:** EC2 instances in private subnets cannot be accessed directly from the internet.
- **Solution:** Use a **bastion host**, which is a public EC2 instance that acts as a secure gateway.

🏗️ Architecture

```
🌐 Internet
    |
[Bastion Host]
Public Subnet
```

```
          SG: allows SSH from specific IPs
            |
            v
   -----------------------
   |                     |
Private EC2 Instances   Private EC2 Instances
SG: allows SSH from bastion host only
Private Subnet
```

- **Bastion host:** EC2 in public subnet with public IP
- **Private EC2:** EC2 in private subnet with no public IP
- **Traffic flow:**

```
Your laptop → SSH → Bastion Host → SSH → Private EC2
```

🔒 Security Group Rules

Bastion Host SG

- **Inbound:** SSH (port 22)
- **Source:** Only your corporate/public IP (not 0.0.0.0/0!)
- **Reason:** Restrict access to prevent unauthorized entry

Private EC2 SG

- **Inbound:** SSH (port 22)
- **Source:** Private IP or Security Group of bastion host
- **Reason:** Only allow traffic originating from the bastion host

> Exam Tip 💡
> Security group referencing another security group is a common exam topic.

⚡ Why Bastion Hosts Are Important

1. **Secure Access:** Private EC2 instances remain inaccessible from the public internet.
2. **Centralized Entry Point:** One controlled instance to manage access.
3. **Audit and Logging:** You can monitor who logs into private instances via the bastion host.

🔄 Steps for Access

1. SSH from your machine to the **bastion host**.
2. SSH from the bastion host to the **private EC2 instance**.
3. Optional: Use SSH agent forwarding for convenience.

📝 Key Points for Interviews

- Must be in **public subnet** with IGW access.
- Restrict **SG rules** for bastion host (only allow specific IPs).
- Private EC2 only allows traffic from bastion host (SG reference).
- Bastion host acts as a **jump server** or **jump box**.
- Often combined with **session manager** in modern setups to avoid exposing SSH ports.

This is a classic **VPC networking exam question**:

> *"How do you securely access an EC2 in a private subnet?"*
> Answer: Bastion host in a public subnet + SG rules.

💻 Bastion Host in Action — SSH into Private EC2 (Interview Notes)

This is the **practical walkthrough** of accessing a private EC2 instance via a bastion host.

🔑 Key Concepts

- **Private EC2 instance:** No public IP, cannot access internet directly, cannot be SSH'd from outside.
- **Bastion host:** Public EC2 in a public subnet that acts as a jump server.
- **SSH flow:**

```
Your Laptop → SSH → Bastion Host → SSH → Private EC2
```

🛠️ Steps to Access Private EC2

1️⃣ Create Key Pair (if not already)

- Key pair required to SSH into EC2 instances.
- Save `.pem` file securely.
- Used both for bastion host and private instance.

2️⃣ Launch Bastion Host

- EC2 instance in **public subnet**.
- Assign **security group** to allow SSH from your IP.
- Enable auto-assign public IP.

3️⃣ Launch Private EC2 Instance

- EC2 instance in **private subnet** (no public IP).
- Security group: allow **SSH only from bastion host's security group**.
- Use the **key pair** created earlier for SSH.

4️⃣ SSH via Bastion Host

1. SSH into bastion host from your laptop:

```
ssh -i "demo-key.pem" ec2-user@<bastion-public-ip>
```

2. Transfer the private key to bastion host for private instance access:

```
nano demo-key.pem    # paste private key content
chmod 400 demo-key.pem
```

3. SSH into private EC2 from bastion host:

```
ssh -i "demo-key.pem" ec2-user@<private-EC2-IP>
```

✅ You are now inside the private EC2 instance.

⚠️ Common Issues

- **Key pair format:** Must be correct; improper newlines cause SSH errors.
- **Permissions:** `.pem` file must have 400 permissions (`chmod 400`).
- **Security groups:** Private EC2 only allows SSH from bastion host, not from the internet.

🌐 Internet Access

- Private EC2 **cannot access the internet** by default.
- Even if you SSH through the bastion, pinging Google fails.
- Solution: Use **NAT Gateway** in a future step to allow outbound internet traffic from private subnet.

🔑 Exam/Interview Tips

1. Bastion host is a **jump server** for private instances.
2. Always put the bastion host in a **public subnet** with IGW access.
3. Restrict SSH access on bastion host SG to specific IPs.
4. Private EC2 SG allows SSH **only from bastion host SG**.
5. Understand the **SSH chaining** concept:

```
Laptop -> Bastion (Public) -> Private EC2
```

Here's a **concise breakdown of NAT instances** and how they fit into your VPC architecture, perfect for exams or interview notes:

🌐 NAT Instances — Overview

NAT = Network Address Translation

- Purpose: Allow **private subnet EC2 instances** to access the **internet** (outbound only).
- NAT **does not allow inbound connections** from the internet to private instances.

🔷 Key Characteristics

1. **Location:**
    - NAT instance must be in a **public subnet**.
    - Private subnets route traffic to it for internet access.
2. **Elastic IP:**
    - NAT instance needs a **fixed public IP** (Elastic IP) for outgoing connections.
3. **Source/Destination Check:**
    - Must **disable this** on NAT instances.

    - Reason: NAT rewrites source IPs for traffic, so the EC2 instance must accept packets not originally destined for it.

4. **Traffic Flow Example:**

Source IP	Destination IP	Routing Device	NAT Translation
10.0.0.20 (private EC2)	50.60.4.10 (public server)	NAT instance	Source rewritten to Elastic IP of NAT
Reply: 50.60.4.10 → NAT IP	NAT forwards to 10.0.0.20	NAT reverses translation	Private EC2 receives response

---

⚠️ Limitations of NAT Instances

- **End of standard support:** December 31, 2020.
- **Availability:** Not highly available by default; need multiple instances per AZ + ASG for resilience.
- **Bandwidth:** Limited by instance size.
- **Management overhead:**
    - Security Groups (inbound/outbound)
    - Monitoring and maintenance
    - Updates and patching

✅ Summary

- NAT instances allow **outbound internet** from private subnets.
- They require:
    - Public subnet placement
    - Elastic IP
    - Disabled source/destination checks
- **Replacement:** NAT Gateways are now recommended — simpler, highly available, and managed by AWS.

💡 **Exam tip:**
If asked, **always prefer NAT Gateway** over NAT Instance unless specifically testing legacy knowledge.

- NAT Gateway = managed, highly available, scalable, no source/destination check needed.
- NAT Instance = manually managed, less resilient, requires careful routing & SG setup.

Here's a clear summary of your **NAT instance setup and testing** for private subnets:

💻 NAT Instance Hands-On Summary

1. Launching the NAT Instance

- **AMI:** Search "NAT" in Community AMIs → choose a **recent AWS-provided NAT AMI**.
- **Instance type:** T2.micro (or as needed).
- **Key pair:** Use an existing key pair (e.g., `demo key pair`).
- **Network:** Deploy in **public subnet** (e.g., `PublicSubnetA`) of your VPC.
- **Security group (NATInstanceSG):**
  - SSH from your IP (optional, for management)
  - HTTP & HTTPS **from the private subnet CIDR** (e.g., `10.0.0.0/16`)

---

2. Disable Source/Destination Check

- NAT instance **must have source/destination check disabled**.
- Reason: NAT rewrites packets, so traffic that's not addressed to the NAT instance itself must be allowed through.

---

3. Routing Private Subnet Traffic

- Go to the **PrivateRouteTable** → **Edit routes** → Add:
  - **Destination:** `0.0.0.0/0`
  - **Target:** NAT instance

This ensures **all internet-bound traffic from private instances** goes through the NAT instance.

---

4. Testing

- SSH into **Bastion host** → then SSH into **private instance**.
- Test connectivity:
  - `ping google.com` → requires **ICMP allowed** in NAT security group
  - `curl example.com` → HTTP/HTTPS allowed from private subnet

---

5. Key Observations

- Private instances gain **outbound internet access** without public IPs.
- NAT instance provides **internet connectivity without exposing private instances**.
- Must configure **security group rules** and **routing correctly** for this to work.

---

6. Next Step

- NAT instances are **legacy**:
  - Less resilient, need manual scaling, instance size affects bandwidth.
  - **NAT Gateway** is the recommended solution → fully managed, highly available, easier to maintain.

💡 **Exam Tip:**

- Understand the difference: NAT instance vs NAT gateway.
- NAT instance → manual, less resilient, needs source/destination check disabled.
- NAT gateway → managed, highly available, no need to manage EC2, elastic IP automatically handled.

Here's a concise breakdown of **NAT Gateways vs NAT Instances** and how NAT Gateways work:

🌐 NAT Gateway Overview

1. Purpose

- Provide **internet access to private subnets**.
- Deployed in **public subnets**.
- Works **through the internet gateway**.
- Private instances send traffic → NAT Gateway → Internet Gateway → Internet.

---

2. Key Features

Feature	NAT Gateway	NAT Instance
Management	Fully managed by AWS	Self-managed (patching, scaling)
High availability	Within a single AZ by default	Depends on instance setup; manual HA needed
Scaling	Auto-scales: 5 Gbps → 100 Gbps	Limited by instance type

Feature	NAT Gateway	NAT Instance
Public IP	Elastic IP (static)	Elastic or public IP manually assigned
Security groups	Not required	Must configure inbound/outbound manually
Use as Bastion host	❌ Cannot	✅ Can
Maintenance	None	You manage OS, updates, rules
Cost	Hourly + data transfer	Hourly + data transfer + EC2 cost
AZ failure	Resilient only within AZ → create multiple NAT Gateways for multi-AZ HA	Must build failover manually

---

3. How It Works

1. NAT Gateway is in **public subnet**, attached to **Elastic IP**.
2. Private subnet route table updated:
    - Destination `0.0.0.0/0` → Target: NAT Gateway
3. Private instances now have **outbound internet access** but **no public IP**.

---

4. High Availability

- NAT Gateway is **redundant within the AZ**.
- To survive an **AZ failure**, deploy **one NAT Gateway per AZ**.
- No need to manually manage failover between AZs — each NAT Gateway handles traffic for its AZ.

---

5. Benefits over NAT Instances

- **Automatic scaling** → handles high throughput automatically.
- **No patching or security groups to manage** → less administration.
- **Better fault tolerance** → multi-AZ deployment is simple.
- **Simpler architecture** → easier for exams and production setups.

💡 **Exam Tip:**

- Use **NAT Gateways** unless you need a **bastion host**.
- For **private subnet internet access**, NAT Gateways are the preferred, highly available solution.

- NAT instances are mostly legacy but may still appear in exam questions to test conceptual understanding.

Perfect! Now your **private subnet EC2 instances have internet access** through the **NAT Gateway**, without needing public IPs. Let me summarize the key steps and concepts from what you just did:

✅ Steps You Took for NAT Gateway Setup

1. **Stopped the old NAT instance**
    - Private subnet route table had a black hole route pointing to the stopped instance.
    - Highlighted why managed services are better: no single point of failure.
2. **Created a NAT Gateway**
    - Name: `DemoNATGW`.
    - Subnet: `PublicSubnetA` (for single-AZ testing; multi-AZ needed for HA).
    - Allocated an **Elastic IP** (mandatory for NAT Gateways in public subnets).
3. **Updated the Private Route Table**
    - Removed the black hole route.
    - Added a new route: `0.0.0.0/0` → `DemoNATGW`.
    - Result: all internet-bound traffic from private subnets now goes through the NAT Gateway.
4. **Waited for NAT Gateway to become active**
    - NAT Gateways can take a minute or two to transition from **pending** → **available**.
5. **Tested connectivity**
    - On the private EC2 instance (via Bastion Host):
        - `curl google.com` → works.
        - `ping google.com` → works.
    - No security group modifications required — NAT Gateway automatically handles traffic.
6. **Optional High Availability**
    - For full multi-AZ resilience:
        - Deploy one NAT Gateway per AZ.

        - Update route tables of private subnets in each AZ to use their local NAT Gateway.
    - Each NAT Gateway handles traffic for its own AZ.

💡 Key Takeaways

- **NAT Gateway vs NAT Instance**
    - NAT Gateway = managed, scalable, no security groups needed, simple to maintain.
    - NAT Instance = deprecated, requires manual patching, configuration, and HA setup.
- **Private subnet benefits**
    - Instances can access the internet for updates or API calls.
    - Still remain **private** (no public IP exposed).
- **Exam tip**
    - If asked: "How do you provide internet access to private subnets?", the answer is usually **NAT Gateway**.
    - NAT Instance may appear as a legacy option.

Absolutely! Let's create **detailed, exam-ready notes** with ASCII diagrams to visualize how **Security Groups (SG) and Network ACLs (NACL) work**, including ephemeral ports. I'll break it down step by step.

🔷 AWS Security Groups and NACLs – Detailed Notes

1️⃣ Overview

- **Security Group (SG)** – Instance-level firewall, **stateful**, allows **only** rules you define. Outbound responses are automatically allowed.
- **Network ACL (NACL)** – Subnet-level firewall, **stateless**, allows **allow & deny** rules. Must configure inbound **and** outbound explicitly.

2️⃣ Traffic Flow: Inbound Request

Scenario: Client → EC2 instance (private subnet)

```
Client ──▶ [Internet / Other subnet]
```

```
          |
          ▼
     +-----------------+
     |  NACL (Subnet)  | ← inbound rules checked first (stateless)
     +-----------------+
          |
          ▼
     +-----------------+
     | Security Group  | ← inbound rules checked next (stateful)
     +-----------------+
          |
          ▼
        EC2 Instance
          |
          ▼ Response
```

Explanation:

1. **NACL inbound:** First firewall → if denied → blocked immediately.
2. **Security Group inbound:** Next firewall → if allowed → traffic reaches instance.
3. **EC2 response:**
    - SG is **stateful** → outbound response automatically allowed.
    - NACL is **stateless** → outbound rules evaluated separately.

---

3 Traffic Flow: Outbound Request

Scenario: EC2 instance → External server

```
EC2 Instance
     |
     ▼
+-----------------+
| Security Group   | ← outbound rules (stateful)
+-----------------+
     |
     ▼
+-----------------+
```

```
| NACL (Subnet)    | ← outbound rules (stateless)
+-----------------+
     |
     ▼
External Server
     |
     ▼ Response
+-----------------+
| NACL inbound     | ← stateless, must allow return traffic
+-----------------+
     |
     ▼
+-----------------+
| Security Group   | ← stateful, automatically allows return traffic
+-----------------+
     |
     ▼
EC2 Instance receives response
```

---

4 NACL Rule Precedence

- Numbered **1** → **32,000** (lower = higher priority)
- **First match wins**
- Default NACL:
  - Allows all inbound and outbound traffic (IPv4)
- Custom NACL:
  - Denies everything by default → must explicitly allow traffic.

Example:

Rule #	Type	Allow / Deny	Source / Destination
100	ALL	ALLOW	10.0.0.0/16
200	ALL	DENY	10.0.0.5/32

→ 10.0.0.5 is **allowed** because rule 100 has higher priority.

---

5 Ephemeral Ports

- Temporary ports chosen by client for return traffic
- Typical ranges:
  - **Linux:** 32,768 → 60,999
  - **Windows:** 49,152 → 65,535
- Server responds to ephemeral port, not a fixed port.

Example Traffic (Client → Server)

```
Client IP: 10.0.1.10
Client ephemeral port: 50105
Server IP: 10.0.2.10
Server port: 3306 (MySQL)

Client ── TCP 50105 ──▶ 3306 Server
Server ── TCP 3306 ──▶ 50105 Client
```

Implications for NACLs:

- Must allow outbound from client to server port (3306)
- Must allow inbound from server to **client ephemeral port range** (1024–65,535)

---

6 Concrete NACL Example – Web → DB

Subnets:

- Web Subnet: 10.0.1.0/24
- DB Subnet: 10.0.2.0/24

Traffic: Web client → DB

```
Web NACL:
  Outbound: TCP 3306 → 10.0.2.0/24 (DB)
  Inbound: TCP 1024-65535 ← 10.0.2.0/24 (DB)

DB NACL:
  Inbound: TCP 3306 ← 10.0.1.0/24 (Web)
  Outbound: TCP 1024-65535 → 10.0.1.0/24 (Web)
```

Why ephemeral ports matter:
When DB responds to the web server, the traffic uses ephemeral ports of the web client → must be allowed explicitly in NACL.

7️⃣ Security Group vs NACL – Quick Reference

Feature	Security Group	NACL
Level	Instance	Subnet
Stateful	✅	❌
Allows	Only allow rules	Allow & Deny rules
Default	Deny inbound, allow all outbound	Allow all inbound/outbound
Rules evaluation	All rules checked	Numbered priority, first match wins
Applies to	Attached instances	All instances in subnet

8️⃣ ASCII Diagram – Full Request Flow (SG + NACL + Ephemeral Ports)

```
[Client] --> [Web Subnet NACL Outbound: TCP 3306]
             --> [Web Security Group Outbound: TCP 3306]
             --> [DB Subnet NACL Inbound: TCP 3306]
             --> [DB Security Group Inbound: TCP 3306]
             --> [DB Instance]


Response from DB:
[DB Instance]
 --> [DB Security Group Outbound: TCP ephemeral port 50105]
 --> [DB NACL Outbound: TCP ephemeral port 1024-65535]
 --> [Web Subnet NACL Inbound: TCP ephemeral port 1024-65535]
 --> [Web Security Group Inbound: automatically allowed]
 --> [Web Client ephemeral port 50105]
```

✅ Exam Tips / Key Points

1. **SG is stateful** → outbound responses automatically allowed.
2. **NACL is stateless** → inbound AND outbound rules must allow traffic.
3. **Ephemeral ports** are required for return traffic, must be allowed in NACL.
4. **Default NACL** = allow all, **custom NACL** = deny all by default.

5. **First match wins** in NACL rules numbering.
6. **SGs cannot deny**, NACLs can.
7. **SG = instance-level firewall**, **NACL = subnet-level firewall**.

🔷 Stateful vs Stateless

1️⃣ Stateful (Security Groups)

- **Tracks connection state** → keeps a "memory" of allowed connections.
- **Inbound rules:** Traffic is evaluated against the rules.
- **Outbound rules:** Responses to **allowed inbound traffic** are automatically allowed — you **don't need an explicit rule**.

Example:

- EC2 inbound SSH (22) allowed from 1.2.3.4 → client connects → response traffic (outbound) back to 1.2.3.4 is automatically allowed.
- You **don't need to allow outbound 22 explicitly** for responses.

Key: Outbound responses are automatically accepted if the inbound connection was allowed.

2️⃣ Stateless (NACLs)

- **Does NOT track connection state** → treats every packet independently.
- **Inbound rules:** Evaluated **every time a packet comes in**.
- **Outbound rules:** Evaluated **every time a packet goes out**.

Implications:

- If inbound allows a connection but outbound does **not allow return traffic**, response will be blocked.
- If outbound allows a request but inbound does **not allow the return**, response will be blocked.

Key: You must explicitly allow **both directions** for connections to succeed.

3️⃣ Quick Mental Rule

Feature	Inbound	Outbound	Memory
Stateful (SG)	Evaluated	Responses auto-allowed	✅ tracks connection
Stateless (NACL)	Evaluated	Evaluated	❌ no memory, each packet independent

---

🔷 ASCII Analogy

```
Stateful (SG):

Client ─▶ [SG inbound allow] ─▶ EC2
  ◀── Response auto allowed ──┘

Stateless (NACL):

Client ─▶ [NACL inbound allow] ─▶ EC2
  ─▶ [NACL outbound allow] ─▶ Response
```

Takeaway:

- SG **scrutinizes inbound** but **outbound responses are automatic**.
- NACL **scrutinizes both inbound and outbound** because it has **no history**.

Perfect! Let's summarize this in **detailed notes with ASCII diagrams** and highlight the key points from your demo so it's easy to visualize.

🔷 Network ACLs (NACLs) vs Security Groups (SGs) – Detailed Notes

1️⃣ Network ACLs (NACLs)

- **Level:** Subnet-level firewall.
- **Stateless:** Each packet is evaluated independently. Return traffic must be explicitly allowed.
- **Rule numbering:** Rules have numbers (1–32,000). **Lower number = higher precedence.** First match wins.
- **Default NACL:**
  - Allows all inbound/outbound traffic by default.
  - Automatically associated with new subnets.
- **Custom NACLs:** You can allow or deny traffic. Deny rules can block traffic, but precedence matters.

2️⃣ Security Groups (SGs)

- **Level:** Instance-level firewall.

- **Stateful:** Return traffic for allowed inbound connections is automatically permitted.
- **Rules:** Only allow rules are needed.
- **Default SG:** Denies all inbound, allows all outbound.

---

3 Demo Scenarios Recap

◆ Default NACL

```
Inbound Rules:   Allow ALL (0-65535)
Outbound Rules:  Allow ALL (0-65535)
```

- Everything is allowed for instances in the associated subnets.

◆ Adding Deny Rule with Precedence

- Deny HTTP inbound from `0.0.0.0/0` **rule number 80**

```
Inbound Rules (sorted by rule number):
  80 - DENY HTTP 0.0.0.0/0
 100 - ALLOW ALL
```

- **Result:** HTTP blocked (timeout) → first match wins, DENY takes precedence.
- Move Deny rule to **rule number 140** (after Allow 100):

```
Inbound Rules (sorted by rule number):
 100 - ALLOW ALL
 140 - DENY HTTP 0.0.0.0/0
```

- **Result:** HTTP works → first match (ALLOW 100) takes effect, DENY 140 ignored.

◆ Stateless Behavior Demonstration

- **Outbound rule DENY all** → Inbound HTTP allowed, but return traffic blocked:

```
Inbound Rules:
 100 - ALLOW HTTP
Outbound Rules:
  100 - DENY ALL
```

- Result: Web page times out → NACL is **stateless**, outbound must also allow the return traffic.

◆ Security Group Stateful Behavior

- Inbound rule allows HTTP (80), outbound rule removed:

```
Inbound Rules:  ALLOW HTTP 0.0.0.0/0
```

```
Outbound Rules: NONE
```

- Result: HTTP still works → SG is **stateful**, outbound return traffic automatically allowed.
- But if the instance initiates traffic (e.g., `curl google.com`), it's blocked because **outbound initiation requires explicit rule**.

---

4 ASCII Diagram: SG + NACL Traffic Flow

```
Internet Client
      │
      │ HTTP Request
      ▼
┌──────────┐
│  NACL    │ (stateless)
│Inbound   │
└──────────┘
      │
      ▼
┌──────────┐
│ Security │ (stateful)
│ Group    │
└──────────┘
      │
      ▼
 EC2 Instance
      │
      │ HTTP Response
      ▼
┌──────────┐
│ Security │
│ Group    │ (stateful)
└──────────┘
      │
      ▼
┌──────────┐
│  NACL    │ (stateless)
```

```
    |Outbound  |
    └──────────┘
        |
        ▼
Internet Client
```

Key Points from Diagram:

- NACL must explicitly allow inbound **and** outbound for connection to work (stateless).
- Security Group only needs inbound rule for incoming traffic; outbound is automatically allowed (stateful).
- NACL rules are evaluated **first match wins** based on rule number.

---

5 Key Takeaways

1. **Precedence Matters:** First match in NACL rules wins.
2. **Statefulness:**
    - SG → stateful → return traffic auto-allowed.
    - NACL → stateless → must allow return traffic.
3. **Use Cases:**
    - SG → fine-grained instance-level control.
    - NACL → broad subnet-level firewall, e.g., block specific IPs.
4. **Default Behavior:**
    - Default NACL → allows all traffic.
    - Default SG → denies inbound, allows outbound.
5. **Troubleshooting:** Always check both NACLs and SGs when traffic is blocked.

Here's a detailed, structured set of notes on VPC Peering, including ASCII diagrams to make the concepts clear.

◆ VPC Peering – Detailed Notes

1 What is VPC Peering?

- **Definition:** VPC peering is a networking connection between two VPCs that allows them to communicate as if they were on the same network.
- **Purpose:** Connect VPCs across:

- **Same account**
- **Different accounts**
- **Different regions (inter-region VPC peering)**

---

2 Important Rules

1. **CIDR non-overlap:**
   - VPC CIDR ranges must **not overlap**.
   - Example:
     - VPC A → 10.0.0.0/16
     - VPC B → 10.1.0.0/16 ✅
     - VPC C → 10.0.5.0/24 ❌ (overlaps with A)
2. **Not transitive:**
   - Peering connections **do not transitively route traffic**.
   - Example:

     `VPC A — peer — VPC B — peer — VPC C`

     - A cannot reach C unless **explicit A–C peering** exists.
3. **Route tables:**
   - You must update the **route tables** of each subnet in the peered VPCs to allow traffic to flow.
4. **Security groups referencing:**
   - You can reference **security groups from the peered VPC** instead of using CIDR IPs.
   - Works **within the same region**, even across accounts.

---

3 ASCII Diagram of VPC Peering

```
+-------------------+
|      VPC A        |
| CIDR: 10.0.0.0/16|
| Subnet A1         |
| Subnet A2         |
+-------------------+
         |
```

```
VPC Peering Connection
          |
 +-------------------+
 |      VPC B        |
 | CIDR: 10.1.0.0/16|
 | Subnet B1         |
 | Subnet B2         |
 +-------------------+
```

Key Notes:

- Each VPC maintains its own CIDR block.
- Traffic between VPCs flows only if **route tables are updated**.
- Security groups can reference **peered VPC security groups** for finer access control.

---

4 Multi-VPC Scenario (Non-Transitive Example)

```
VPC A 10.0.0.0/16 — peer — VPC B 10.1.0.0/16 — peer — VPC C 10.2.0.0/16


- A ↔ B ✅
- B ↔ C ✅
- A ↔ C ❌ (requires separate peering)
```

- Even though B connects both A and C, A cannot reach C without an explicit peering connection.

---

5 Best Practices

- Avoid overlapping CIDRs to prevent routing conflicts.
- Update route tables immediately after peering creation.
- Use **security group references** instead of IP-based rules for cross-VPC security.
- For inter-region VPC peering, be aware of **data transfer costs**.

---

6 Summary

Feature	VPC Peering
Connectivity	VPC ↔ VPC
Across Accounts	✅
Across Regions	✅
Transitive	❌

Feature	VPC Peering
CIDR Overlap Allowed	❌
Route Table Updates	Required
Security Group Reference	✅ (within same region)

Here's a detailed set of notes from your hands-on VPC peering exercise, with ASCII diagrams and key points highlighted for clarity:

🔷 VPC Peering – Hands-On Notes

1️⃣ Scenario Setup

- **Goal:** Connect two VPCs so instances can communicate:
  - **Demo VPC** → contains the BastionHost with HTTP server (`hello world`)
  - **Default VPC** → contains a newly launched EC2 instance
- **Observation before peering:**

  ```
  Demo VPC instance IP: 10.0.0.x
  Default VPC instance IP: 172.31.36.159
  ```

  - Attempting `curl 10.0.0.x` from Default VPC → **Timeout**
  - Reason: Different VPCs are **isolated networks by default**

2️⃣ Create VPC Peering Connection

1. Navigate to **VPC Peering Connections** in AWS console.
2. Click **Create Peering Connection**:
   - **Requester VPC:** Demo VPC
   - **Acceptor VPC:** Default VPC
   - Ensure **CIDR blocks do not overlap**
3. Connection initially in **Pending Acceptance** state.
4. Accept the request (if same account) → Status becomes **Active**

3️⃣ Update Route Tables

Important: VPC peering does **not automatically update route tables**.

1. **Demo VPC Route Table:**
   - Add route:

```
Destination: Default VPC CIDR (172.31.0.0/16)
Target: Demo Peering Connection
```

2. **Default VPC Route Table:**
   - Add route:

     ```
     Destination: Demo VPC CIDR (10.0.0.0/16)
     Target: Demo Peering Connection
     ```

4️⃣ Test Connectivity

- From **Default VPC EC2 instance**:

  `curl 10.0.0.x → returns "hello world"`

- From **Demo VPC instance** (optional):

  `curl 172.31.36.159 → returns "response"`

- ✅ Successful two-way communication established

5️⃣ ASCII Diagram

```
          Demo VPC (10.0.0.0/16)
+-----------------------------+
| BastionHost                 |
| Private IP: 10.0.0.x        |
| HTTP Server: 80             |
+-----------------------------+
            |
    VPC Peering Connection
            |
          Default VPC (172.31.0.0/16)
+-----------------------------+
| EC2 Instance                |
| Private IP: 172.31.36.159   |
+-----------------------------+
```

- **Route Tables updated** so traffic knows to use the **peering connection** to reach the other VPC.
- Communication is now **bidirectional**.

---

6 Key Takeaways

1. **VPCs are isolated by default:**
    - Different CIDRs, cannot communicate without peering.
2. **Peering is not transitive:**
    - Each pair of VPCs needs its own peering connection.
3. **Routes must be updated in both VPCs** to allow traffic to flow.
4. **CIDR blocks cannot overlap** for a valid peering connection.
5. **Works across accounts and regions** (with some limitations for cross-region).
6. **Security groups:**
    - Ensure inbound/outbound rules allow traffic from the **peered VPC CIDR or security group**.

---

7 Before vs After Peering

Action	Before Peering	After Peering
`curl` from Default → Demo	Timeout	"hello world"
Route tables	No routes to other VPC	Routes added for peering connection
Security	EC2 SG allows HTTP	Still required, SG + route needed
Peering connection	N/A	Active

This workflow is **essential** for exams and real-world VPC networking, because it highlights both **peering setup** and **route table updates**, which are common points of confusion.

Here's a detailed breakdown of your VPC Endpoints lecture with diagrams and key concepts:

◆ VPC Endpoints – Private Access to AWS Services

1 Problem Statement

- Many AWS services (S3, DynamoDB, SNS, CloudWatch) are **publicly accessible** by default.
- EC2 instances in **private subnets** must go through:
    - **NAT Gateway → Internet Gateway → Public AWS service**
- Issues with this approach:
    - Traffic leaves AWS public network → security concern.
    - Costs more → NAT Gateway hourly + data transfer.

    - Extra hops → latency and complexity.

---

2 VPC Endpoints Concept

- Allow **private connectivity** to AWS services **without using the internet**.
- Traffic stays within the **AWS private network**.
- Benefits:
    - Reduced cost (no NAT needed for supported services).
    - Improved security (no public internet exposure).
    - High availability and automatic scaling.
- Powered by **AWS PrivateLink** for interface endpoints.

---

3 Types of VPC Endpoints

A. Interface Endpoint

- **Powered by PrivateLink.**
- **Provisioned as an ENI (Elastic Network Interface)** in your VPC.
- Requires **security groups** to control access.
- Can access **most AWS services**.
- **Cost:** per hour + per GB processed.
- Example:

```
Private Subnet EC2
+-----------------+
| EC2 Instance    |
| Private IP      |
+-----------------+
        |
        v
+-----------------+
| Interface ENI   | <-- VPC Endpoint (PrivateLink)
| Private IP      |
+-----------------+
        |
        v
```

```
AWS Service (e.g., CloudWatch, SNS)
```

B. Gateway Endpoint

- Used **only for S3 and DynamoDB**.
- Provisioned as a **gateway**; added to a **route table**.
- **No ENI**, **no security groups**, **no cost**.
- Scales automatically.
- Example:

```
Private Subnet EC2
+-----------------+
| EC2 Instance    |
| Private IP      |
+-----------------+
        |
        v
+-----------------+
| Route Table     |
| Target: Gateway |
+-----------------+
        |
        v
AWS Service (S3 or DynamoDB)
```

- Simple setup → just **update the route table** with the VPC endpoint as a target.

---

4 Interface vs Gateway Endpoint

Feature	Interface Endpoint	Gateway Endpoint
Supported Services	Most AWS services	Only S3 & DynamoDB
Provisioning	ENI + Security Groups	Route table target
Cost	Paid per hour + per GB	Free
Use Cases	Access from on-prem, cross-VPC	Default choice for S3/DynamoDB
Scalability	Automatically scales	Automatically scales
Network	Private IP access	Route-based access

Key exam tip:

- For **S3 or DynamoDB**, use **Gateway Endpoint** (simpler, free).

- Use Interface Endpoint for other services or advanced scenarios (on-premises or cross-VPC).

---

5 Architecture – Example Diagram

```
Public Subnet
+-------------------+
| NAT Gateway        |
+-------------------+
        |
        v
Internet Gateway (if using public AWS service)

Private Subnet
+-------------------+
| EC2 Instance       |
| Private IP         |
+-------------------+
        |
        +-------------------+
        |                   |
        v                   v
 Interface Endpoint    Gateway Endpoint
(PrivateLink ENI)     (Route Table Target)
        |                   |
        v                   v
  CloudWatch / SNS      S3 / DynamoDB
```

- Traffic **never leaves AWS network**.
- Eliminates NAT Gateway for supported services → cost-efficient.
- Provides security and private connectivity.

---

6 DNS & Route Table Notes

- Make sure **DNS resolution** is enabled in the VPC.
- Update **route tables** for gateway endpoints (S3/DynamoDB).
- Interface endpoints are accessed via **private IP addresses** (ENI).

✅ **Summary:**

- VPC endpoints provide **private access to AWS services**.
- **Interface Endpoint:** ENI + security group → any service.
- **Gateway Endpoint:** Route table → S3/DynamoDB → free & simple.
- **Benefits:** Security, cost reduction, high availability, no internet required.

Here's a detailed set of **notes and ASCII diagrams** for your VPC Endpoints hands-on demonstration:

🔷 Practice with VPC Endpoints – Hands-On

1️⃣ Initial Setup

- We have:
  - **Bastion Host** (public EC2)
  - **Private EC2 Instance** (private subnet)
- Goal: Access **Amazon S3** from the private EC2 **without internet access**.

2️⃣ Step 1: Attach an IAM Role

- For S3 access, the EC2 instance must have a role with permissions.
- Role created:
  - Name: `DemoRoleEC2-S3ReadOnly`
  - Permissions: `AmazonS3ReadOnlyAccess`
- Result: EC2 can list S3 buckets using AWS CLI.

```
$ aws s3 ls
```

- Currently works because private EC2 **has internet access via NAT gateway**.

3️⃣ Step 2: Remove Internet Access

- Modify **private subnet route table**:
  - Remove the NAT Gateway route.
- Effect:
  - EC2 **cannot access internet** anymore.
  - Commands like `curl google.com` or `aws s3 ls` fail.

```
Private Subnet Route Table (before):
Destination     Target
0.0.0.0/0       NAT Gateway

Private Subnet Route Table (after removing NAT):
Destination     Target
0.0.0.0/0       <removed>
```

---

4 Step 3: Create a VPC Endpoint

A. Interface Endpoint

- Creates an **ENI** in private subnet.
- Assign a **security group** to control access.
- Can connect to **any AWS service**.
- Steps:
    1. Select **VPC**
    2. Select **subnets** to deploy ENI
    3. Assign **security group**
    4. Enable **DNS resolution** in VPC

```
Private EC2 Instance
+-------------------+
| EC2 Private IP    |
+-------------------+
        |
        v
+-------------------+
| Interface ENI     |
| Security Group    |
+-------------------+
        |
        v
AWS Service (CloudWatch, SNS, etc.)
```

B. Gateway Endpoint (Amazon S3 / DynamoDB)

- Add a **route in route table** pointing to the endpoint.
- No ENI or security groups required.
- Free and automatically scales.
- Steps:
    1. Choose **VPC** → demo VPC
    2. Update **route table** → private route table
    3. Select **full S3 access**
    4. Create endpoint

```
Private Subnet
+-------------------+
| EC2 Instance      |
+-------------------+
        |
        v
+-------------------+
| Route Table       |
| Target: S3 Gateway|
+-------------------+
        |
        v
Amazon S3
```

---

5 Step 4: Verify Endpoint Access

- Try accessing S3:

```
$ aws s3 ls --region eu-central-1
```

- Works even though **internet access is gone**.
- CLI must be set to the **correct region** for VPC Endpoint.
- Access to internet (curl google.com) still fails — expected.

---

6 Key Points

1. **VPC endpoints allow private connectivity** to AWS services without NAT or internet gateway.

2. **Interface Endpoint**:
    - Creates **ENI** + security group.
    - Works for most AWS services.
    - Paid service.
3. **Gateway Endpoint**:
    - Route table target.
    - Only for **S3 & DynamoDB**.
    - Free, scales automatically.
4. **Route Table & DNS**:
    - For gateway endpoint, update **route table**.
    - Ensure **DNS resolution enabled** for interface endpoints.
5. Region matters in CLI commands to hit the private endpoint.

---

7 ASCII Diagram – Full Example

```
          Public Subnet
+---------------------+
| Bastion Host EC2    |
| Public IP           |
+---------------------+
           |
           | SSH
           v
      Private Subnet
+---------------------+
| Private EC2         |
| Private IP          |
+---------------------+
           |
           | Route Table → Gateway Endpoint
           v
        +---------+
        | S3 Bucket|
```

```
        +---------+

(No NAT gateway needed, no internet access required)
```

💡 **Exam Tip:**

- **Private S3/DynamoDB access** → always use **Gateway Endpoint** if possible.
- **Other AWS services** → use **Interface Endpoint**.
- Check **route tables and DNS** if private access doesn't work.

Here's a **structured set of notes with diagrams** for your VPC Flow Logs lecture:

🔷 VPC Flow Logs – Overview

VPC Flow Logs **capture IP traffic metadata** for a VPC, subnet, or ENI.

Purpose:

- Monitor and troubleshoot network connectivity.
- Analyze traffic patterns.
- Detect suspicious behavior (port scans, attacks, etc.).

Supported Targets:

- VPCs
- Subnets
- Elastic Network Interfaces (ENIs)

Supported Services:

- AWS managed services like ELB, RDS, ElasticCache, Redshift, Workspaces, NAT Gateway, Transit Gateway.

Destinations:

- Amazon S3
- CloudWatch Logs
- Kinesis Data Firehose

1️⃣ Flow Log Contents

A typical flow log record contains:

Field	Description
version	Format version of the log
account ID	AWS account ID
interface ID	ENI ID of the interface
srcaddr	Source IP address
dstaddr	Destination IP address
srcport	Source port
dstport	Destination port
protocol	IP protocol (TCP/UDP/ICMP)
packets	Number of packets
bytes	Number of bytes
start	Start timestamp
end	End timestamp
action	`ACCEPT` or `REJECT`
logstatus	Status of logging (OK, NODATA, SKIPDATA)

Key: `action` helps troubleshoot connectivity issues.

---

2 Flow Log Analysis – Security Groups vs NACLs

Scenario	Meaning
Inbound **REJECT**	Could be Security Group or NACL
Inbound **ACCEPT**, Outbound **REJECT**	NACL issue (SG is stateful)
Outbound **REJECT**	Could be SG or NACL
Outbound **ACCEPT**, Inbound **REJECT**	NACL issue

Note: Security Groups are **stateful** → if inbound is allowed, return traffic is automatically allowed.

---

3 Use Cases

A. CloudWatch Logs

- Flow logs → CloudWatch
- Create **Metric Filters** for:
  - SSH traffic spikes
  - RDP attempts
- Trigger **Alarms** → send notifications via SNS

```
Flow Logs
   |
   v
CloudWatch Logs
```

```
   |
   v
Metric Filter -> Alarm -> SNS Notification
```

B. S3 + Athena

- Store logs in **S3**
- Query with **Athena** (SQL)
- Visualize with **QuickSight**

```
Flow Logs
   |
   v
S3 Bucket
   |
   v
Athena (SQL Queries)
   |
   v
QuickSight (Visualization)
```

C. Contributor Insights

- Identify top N IPs consuming network traffic
- Helps detect anomalies and high-usage patterns

---

4 Permissions for VPC Flow Logs

VPC Flow Logs need an **IAM service role** to send logs:

Required permissions:

- `logs:CreateLogGroup`
- `logs:CreateLogStream`
- `logs:PutLogEvents`

These permissions allow the VPC Flow Logs service to publish logs to CloudWatch Logs.

---

5 ASCII Diagram – Flow Log Architecture

```
          VPC / Subnet / ENI
+----------------------------+
```

```
| Flow Logs Captured         |
|  srcaddr / dstaddr         |
|  srcport / dstport         |
|  protocol / packets / bytes|
+----------------------------+
           |
           v
   ---------------------------
   | Logging Destination      |
   |--------------------------|
   | CloudWatch Logs          |
   | S3 + Athena              |
   | Kinesis Data Firehose    |
   ---------------------------
           |
           v
   Analytics / Monitoring
   ---------------------
   | Metric Filters      |
   | Contributor Insights|
   | Alarms / Notifications|
   | Visualizations (QS) |
   ---------------------
```

✅ Key Takeaways

1. Flow logs **capture metadata only**, not the content of packets.
2. Can be used at **VPC, Subnet, or ENI level**.
3. Essential for troubleshooting **SG vs NACL issues**.
4. **CloudWatch + Metrics + Alarms** → real-time monitoring.
5. **S3 + Athena + QuickSight** → historical analysis & visualization.
6. Requires **IAM role permissions** for log delivery.

Perfect! Let's summarize your **VPC Flow Logs hands-on demo** step by step, so you have a clean, structured reference for practice or exam prep.

🔷 VPC Flow Logs Hands-On Summary

1️⃣ Creating a Flow Log

Steps:

1. Go to **VPC → Flow Logs → Create Flow Log**
2. Name your flow log, e.g., `DemoS3FlowLog`
3. Choose **Filter Type**:
   - `ACCEPT` – only successful traffic
   - `REJECT` – only denied traffic (useful for troubleshooting)
   - `ALL` – all traffic
4. Set **Aggregation Interval**:
   - `1 min` – faster updates, more records, more cost
   - `10 min` – typical for production, lower cost
5. Choose **Destination**:
   - Amazon S3 (for analytics)
   - CloudWatch Logs (for monitoring, metrics, alarms)
   - Kinesis Data Firehose (streaming to another system)

2️⃣ Flow Log to Amazon S3

Steps:

1. Create an S3 bucket, e.g., `demo-stephane-vpc-flow-logs-v2`
2. Copy the **bucket ARN**
3. Paste it in the flow log destination
4. AWS automatically creates a **resource-based policy** to allow VPC to write logs
5. Create the flow log → traffic is now stored in S3

Notes:

- S3 is best for **long-term storage** and batch analytics.
- Example path for logs:

```
s3://demo-stephane-vpc-flow-logs-v2/AWSLogs/<account-id>/vpcflowlogs/<region>/...
```

---

3 Flow Log to CloudWatch Logs

Steps:

1. Create **IAM Role** for VPC Flow Logs:
   - Service Principal: `vpc-flow-logs.amazonaws.com`
   - Policy: `CloudWatch Logs Full Access`
2. Create **Log Group**, e.g., `VPCFlowLogs`
3. Create flow log → associate with **ENIs** in the VPC
4. Logs will appear under **Log Streams** corresponding to each ENI

Notes:

- CloudWatch Logs is best for **real-time monitoring**.
- You can create **metric filters** to alert on unusual traffic (e.g., SSH/RDP attempts).

---

4 Inspecting Flow Logs

- Each log shows traffic for an ENI:
  - `srcaddr` → source IP
  - `dstaddr` → destination IP
  - `action` → ACCEPT / REJECT
  - Protocol, ports, bytes, packets, timestamps
- Detect suspicious traffic, e.g., port scans or unauthorized access

---

5 Querying Flow Logs with Athena

Steps:

1. Set **Query Result Location** in Athena → S3 bucket for results, e.g., `demo-athena-stephane-v2`
2. Create **Database & Table** in Athena for your flow logs:

```
CREATE EXTERNAL TABLE vpc_flow_logs (
  version int,
  account_id string,
  interface_id string,
```

```
  srcaddr string,
  dstaddr string,
  srcport int,
  dstport int,
  protocol int,
  packets bigint,
  bytes bigint,
  start bigint,
  end bigint,
  action string,
  log_status string
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ' '
LOCATION 's3://<your-vpc-flow-log-bucket>/AWSLogs/<account-id>/vpcflowlogs/
<region>/';
```

3. Add **partitions** for date to read daily logs:

```
ALTER TABLE vpc_flow_logs ADD PARTITION (year='2021', month='10', day='06')
LOCATION 's3://.../2021/10/06/';
```

4. Query logs:

```
SELECT * FROM vpc_flow_logs WHERE action='REJECT';
```

- Can group by IP, ENI, protocol, etc.
- Useful for **batch analysis** of attackers or traffic patterns

---

6 Key Observations

- **CloudWatch Logs** → real-time alerting and monitoring
- **S3 + Athena** → batch analytics and historical queries
- **Metric Filters / Alarms** → trigger SNS notifications on unusual patterns
- **Cost Management**:
  - Use aggregation intervals wisely (1 min → higher cost)
  - Delete logs when not needed

7️⃣ Architecture Overview

```
        +----------------+
        | VPC / Subnet / |
        | ENI Traffic    |
        +--------+-------+
                 |
    +-----------+-----------+
    | Flow Logs Capture     |
    +-----------+-----------+
    | Destination Options:  |
    | 1. CloudWatch Logs    |
    | 2. S3 + Athena        |
    | 3. Kinesis Firehose   |
    +----------------------+
                 |
     +----------+----------+
     | Analytics / Monitoring |
     +----------------------+
     | - Metric Filters      |
     | - Alarms / SNS        |
     | - SQL Queries (Athena)|
     | - Visualization (QS)  |
     +----------------------+
```

✅ Tips for Practice / Exam

1. Use **REJECT** filter if troubleshooting connectivity issues.
2. CloudWatch → quick monitoring, S3 → historical analysis.
3. Remember to **create IAM roles** for flow logs.
4. Aggregation interval affects cost and log volume.
5. Use Athena to **query, partition, and analyze** flow logs.

Perfect! Let's break down your **Site-to-Site VPN lecture** into a clear, structured summary so it's easy to remember for practice or exams.

◆ AWS Site-to-Site VPN Overview

1 Purpose

- Connect your **AWS VPC** to your **on-premises corporate network**.
- Traffic flows over the **public internet** but is **encrypted**.
- This allows private communication between your AWS resources and your data center.

---

2 Key Components

Component	Description
Virtual Private Gateway (VGW)	AWS-side VPN concentrator attached to the VPC. Handles VPN connections. Can customize ASN (Autonomous System Number).
Customer Gateway (CGW)	On-premises device or software. AWS uses its IP (public or NATed) to connect.
VPN Connection	Encrypted connection between VGW and CGW. Supports static or dynamic routing.

Notes:

- If CGW is behind NAT → use NAT's public IP (with NAT-T enabled).
- VGW is always in AWS and attached to the VPC.

---

3 Routing & Security Considerations

1. **Route Propagation**
    - VPN will **not work** until you enable **route propagation** on the VPC route tables.
    - This allows subnets in your VPC to learn about the on-premises network.
2. **Security Groups**
    - Even if the VPN is up, traffic may fail if **security groups** don't allow the correct protocols.
    - Example: To ping (ICMP) from on-prem → ensure ICMP inbound is allowed.

---

4 AWS VPN CloudHub

- **Purpose:** Connect multiple customer networks (data centers) to a single VPC VGW.
- **Topology:** Hub-and-spoke (VGW is the hub, CGWs are spokes).
- **Traffic:** Encrypted over public internet.
- **Benefits:**

    - Allows multiple on-prem sites to communicate via a single VPC.
    - Low-cost alternative to dedicated interconnects.
- **Setup:**
    - Create multiple site-to-site VPN connections to the same VGW.
    - Enable **dynamic routing**.
    - Configure **route tables** for each subnet and VPN.

---

5 Exam Tips

- VPN connections require **route propagation** in route tables.
- VPN traffic is encrypted, but it still traverses the **public internet**.
- ICMP or other protocols must be allowed in **security groups**—don't confuse VPN setup with SG configuration.
- VPN CloudHub = multiple on-prem networks connecting to one VPC hub.

---

6 Architecture Diagram (Simplified)

```
On-Prem Data Center 1  --->|             |<--- On-Prem Data Center 2
Customer Gateway 1         |              |  Customer Gateway 2
                           | VGW (AWS)    |
                           |              |
                           VPC Subnets & EC2
```

- Single VGW handles multiple VPN connections (CloudHub)
- All traffic is encrypted over the public internet
- Subnets route propagated for proper connectivity

Perfect! Let's summarize **AWS Direct Connect (DX)** in a structured way so it's easy to digest for hands-on use and exam prep.

◆ AWS Direct Connect Overview

1 Purpose

- Provides a **dedicated private connection** from your **on-premises data center** to **AWS VPC**.

- Traffic **does not traverse the public internet**, so it is more **consistent**, **secure**, and **high-bandwidth**.

---

2 Components

Component	Description
Direct Connect Location	Physical AWS facility where you terminate your connection.
Customer Router	On-premises device that connects to AWS DX.
Direct Connect Connection	Physical Ethernet connection, can be **dedicated** or **hosted**.
Virtual Interface (VIF)	Logical connection over DX; two types: **private VIF** (to VPC) and **public VIF** (to AWS public services).
Virtual Private Gateway (VGW)	AWS-side device that private VIF connects to.
Direct Connect Gateway (DXGW)	Connects multiple VPCs across regions to a single DX connection.

---

3 VIF Types

VIF Type	Purpose
Private VIF	Access private AWS resources like EC2 in your VPC. Connects to VGW.
Public VIF	Access AWS public services like S3, Glacier. Connects directly to AWS endpoints.

---

4 Use Cases

- High-throughput data transfers (large datasets).
- Low-latency or real-time applications.
- Consistent networking without internet variability.
- Hybrid cloud environments (on-prem + AWS).
- Supports **IPv4 and IPv6**.

---

5 Connection Types

Type	Details
Dedicated Connection	1, 10, 100 Gbps physical Ethernet port. Longer setup (~1 month). High reliability.
Hosted Connection	Provided by a DX partner. Flexible bandwidth (15 Mbps → 10 Gbps). Can scale on demand.

Exam tip: Direct Connect setup takes **weeks**. For urgent data transfers (<1 week), DX is **not a feasible option** unless already provisioned.

6️⃣ Encryption

- **Direct Connect traffic is not encrypted** by default.
- You can combine DX with a **VPN (IPsec)** to encrypt traffic over the private connection.

7️⃣ Multi-VPC & Multi-Region

- Use **Direct Connect Gateway (DXGW)** to connect **multiple VPCs across regions** to a single DX connection.
- Private VIF → DXGW → VGW in each region/VPC.

8️⃣ Resiliency Options

Level	Description
High Resiliency	Two DX connections in **two separate DX locations**. Provides backup if one location fails.
Maximum Resiliency	Four DX connections across **two DX locations**, terminating on separate devices. Fully redundant for critical workloads.

Exam tip: AWS may ask: *"You need maximum availability for a critical workload"*. This requires **2 DX locations × 2 connections each = 4 total connections**.

9️⃣ Simplified Architecture Diagram

```
On-Prem Data Center
  └─ Customer Router
       └─ Private VIF ──> Direct Connect Location ──> VGW ──> VPC
       └─ Public VIF  ──> Direct Connect Location ──> AWS Public Endpoints


Multiple VPCs (multi-region)
  └─ DXGW
       └─ Private VIF ──> VGW (Region 1 VPC)
       └─ Private VIF ──> VGW (Region 2 VPC)
```

✅ Key Exam Takeaways

1. DX = private, high-bandwidth, low-latency, but **not encrypted**.
2. Combine DX + VPN for encryption.
3. DX takes **weeks to set up**, so not for urgent transfers.
4. Use **DXGW** to connect multiple VPCs across regions.

5. Resiliency requires **multiple connections & locations**.

Exactly! This is a classic **hybrid connectivity exam scenario**. Let me summarize it concisely:

🔷 Primary + Backup Hybrid Architecture

Scenario

- You have a **corporate data center** connected to your **AWS VPC**.
- Primary connection is **Direct Connect (DX)** → private, high bandwidth, low latency, but costly.
- You want **resiliency** in case DX fails.

Backup Options

1. **Secondary Direct Connect**
    - Fully private, same benefits as primary.
    - Expensive (requires another DX setup).
    - Provides **high resiliency**.
2. **Site-to-Site VPN (over public internet)**
    - Encrypted IPsec tunnel over the public internet.
    - Cheaper than a secondary DX.
    - Provides **failover** if DX goes down.
    - Slightly higher latency, but ensures connectivity.

Typical Setup

- **Primary:** Direct Connect → VPC
- **Backup:** Site-to-Site VPN → VGW → VPC
- Configure **route priority or BGP** so that VPN only activates if DX fails.

Key Exam Takeaways

- **Cost vs. resiliency trade-off:** DX is expensive; VPN is cheap for failover.
- **Hybrid architecture is common:** Primary DX + VPN backup is a standard pattern.
- **Always think "private + encrypted fallback":** DX = private, VPN = encrypted over public internet.

💡 **Tip:** In exam questions, if they ask *"low latency + failover but cost-sensitive"*, the answer is usually **DX primary + VPN failover**, not two DX lines.

Perfect! This is a great overview of **AWS Transit Gateway**—let's break it down in a structured way so it's easy to remember for exams:

🔷 What is a Transit Gateway (TGW)?

- A **regional, fully managed AWS service** to simplify VPC connectivity.
- Acts as a **hub** for a hub-and-spoke network topology:
  - Multiple VPCs
  - On-premises networks (via Direct Connect or VPN)
  - VPN connections
  - Direct Connect gateways
- Eliminates the need for **VPC peering mesh**, which becomes complex with many VPCs.
- Supports **IP multicast**—unique in AWS networking.

🔷 Key Features & Use Cases

Feature	Explanation
Transitive Routing	VPCs connected to TGW can communicate with each other without direct peering.
Centralized Hybrid Connectivity	Connect on-premises via **Direct Connect** or **site-to-site VPN** to multiple VPCs at once.
Cross-account & cross-region	Use **RAM (Resource Access Manager)** to share TGW across accounts; peering allows cross-region connectivity.
VPN ECMP	Use **Equal-Cost Multi-Path routing** to increase VPN bandwidth: multiple tunnels = higher throughput.
Direct Connect Sharing	One DX can serve multiple VPCs and accounts via TGW.
Route Control	Custom **route tables per TGW attachment** to control which VPC or VPN can talk to others.

🔷 Transit Gateway + VPN Throughput Example

- Site-to-site VPN directly to a VPC:
  - 1 VPN connection → 2 tunnels → max 1.25 Gbps.
- Site-to-site VPN to a **TGW**:

- 1 VPN connection → 2 tunnels → 2.5 Gbps (thanks to ECMP).
- Add more VPN connections → increase total throughput proportionally.

🔷 Transit Gateway + Direct Connect Sharing

- One Direct Connect connection → Direct Connect Gateway → TGW → multiple VPCs (even across accounts).
- **Benefit:** cost-efficient sharing of a single DX connection, no need for individual DX per VPC.

✅ Exam Tip:

1. If you see **"transitive connectivity across VPCs, VPNs, and DX"**, the answer is almost always **Transit Gateway**.
2. **IP multicast support?** Only TGW supports it.
3. For **high VPN throughput** or multi-VPC VPN, think **ECMP + TGW**.
4. **Route tables in TGW** are your control plane for security and connectivity.

Exactly! **VPC Traffic Mirroring** is a really powerful tool for **network monitoring and security inspection**. Let's break it down clearly:

🔷 What is VPC Traffic Mirroring?

- **Purpose:** Capture and inspect network traffic in your VPC **without affecting the original source**.
- **How it works:**
  - Mirrors traffic from **source ENIs** (Elastic Network Interfaces).
  - Sends it to **target ENIs** or **Network Load Balancers** running security or monitoring appliances.
- **Non-intrusive:** The original instance continues working normally; traffic is just duplicated and sent to your monitoring system.

🔷 Key Components

Component	Role
Source ENI	The network interface whose traffic you want to mirror.
Target ENI / NLB	Where mirrored traffic is sent for analysis. Can be EC2 instances running IDS/IPS or security appliances.

Component	Role
Filter (optional)	Capture only a subset of traffic (e.g., specific ports, protocols, or IPs).

🔷 Example Architecture

1. **Source EC2 instance:** Handles your normal traffic.
2. **Target (via NLB + EC2 Auto Scaling Group):** Runs traffic inspection software.
3. **Traffic mirroring session:** Captures inbound and outbound traffic from the source and duplicates it to the target.
4. **Optional filters:** Capture only traffic relevant for analysis (reduces cost and noise).

This allows **threat monitoring, content inspection, and network troubleshooting** at scale.

🔷 Use Cases

- Detect malicious traffic or anomalies in your VPC.
- Inspect network traffic for compliance/security audits.
- Troubleshoot networking issues without interrupting production workloads.

✅ Important Notes

- Source and target must be in **the same VPC** or connected via **VPC peering**.
- Traffic mirroring can impact **network bandwidth**—so plan for sufficient capacity on your target ENIs.
- Works for **multiple sources**, so you can mirror traffic from many EC2 instances into one or multiple monitoring targets.

Perfect! This is a very **practical lecture on AWS networking costs**, and there are several **key takeaways for the exam**. Let me summarize and organize it clearly for you.

🔷 AWS Networking Costs Overview

1. EC2 Traffic

Scenario	Cost per GB	Notes
Inbound traffic (into EC2)	Free	Always free
Within same AZ, private IP	Free	Uses AWS internal network
Across AZs, private IP	~$0.01 per GB	Uses AWS backbone, cheaper than public IP
Across AZs, public/elastic IP	~$0.02 per GB	Traffic goes out and back into AWS

Scenario	Cost per GB	Notes
Across regions	~$0.02 per GB	Inter-region traffic is more expensive

Takeaways:

- Prefer **private IPs** for inter-instance communication.
- Keep latency-sensitive and high-volume communication **within the same AZ** if cost optimization is important.
- Balance cost with **high availability** if using same AZ.

---

2. Egress vs Ingress

- **Ingress:** Traffic into AWS → usually **free**.
- **Egress:** Traffic out of AWS → you **pay**.
- Architectural tip: Keep traffic inside AWS (same AZ/region) to minimize egress costs.

Example:

- Query 100 MB of data in AWS, process it inside AWS, return 50 KB results → huge savings.
- Moving the processing outside AWS → pay for the full 100 MB egress.

---

3. Direct Connect

- Use a **co-located Direct Connect location** to reduce egress cost.
- Offers **lower cost and consistent throughput** compared to public internet.

---

4. S3 Data Transfer

Scenario	Cost per GB (US)	Notes
Data upload to S3 (ingress)	Free	Always free
Download from S3 (internet)	~$0.09	Egress traffic
S3 → CloudFront	Free	Within AWS network
CloudFront → Internet	~$0.085	Slightly cheaper than S3 direct
Cross-region replication	~$0.02	Charged per GB replicated
S3 Transfer Acceleration	+$0.04–$0.08	Extra speed, extra cost

Tip: Use **CloudFront** for caching and cost savings on repeated access.

---

5. NAT Gateway vs VPC Endpoint

Approach	Cost Notes
NAT Gateway + Internet Gateway	$0.045/hour for NAT + $0.045/GB processed + data transfer cost (cross-region $0.09/GB)

Approach	Cost Notes
VPC Gateway Endpoint (S3)	$0 + $0.01/GB

Takeaways:

- Prefer **VPC endpoints** for S3 access from private subnets.
- NAT Gateways are convenient but more expensive.

✅ Key Exam Tips

1. **Private IPs for intra-region traffic** → cheaper and faster.
2. **Same AZ communication** → no cost, but less resilient.
3. **Keep heavy computation inside AWS** → minimize egress.
4. **Use VPC endpoints for S3 access** → cheaper and private.
5. **CloudFront for public data distribution** → cheaper and faster than S3 direct.
6. **Cross-region replication** → $0.02/GB, plan for data transfer costs.

Perfect! Let's summarize **AWS network protection options** and where the **AWS Network Firewall** fits in.

🔷 AWS Network Security Tools Overview

Service	Scope / Layer	Key Use Case
Network ACLs (NACLs)	Subnet-level, stateless	Control inbound/outbound traffic at subnet level; simple allow/deny rules
Security Groups	Instance-level, stateful	Acts like a virtual firewall for EC2 instances; allow/deny based on IP, port, protocol
AWS WAF	Application Layer (L7)	Protect web apps against HTTP/HTTPS attacks like SQL injection or XSS
AWS Shield / Shield Advanced	Network Layer (DDoS)	Protect against DDoS attacks; Shield Advanced gives additional protections and reporting
AWS Firewall Manager	Multi-account	Centralized management for WAF, Shield, and security policies across accounts

---

🔷 AWS Network Firewall

Scope: Entire VPC, layer 3 to layer 7 traffic

Use Cases:

- Inspect **all inbound/outbound traffic** in the VPC
- Protect traffic from:

    - Internet
    - Peered VPCs
    - Direct Connect
    - Site-to-site VPNs

How it works:

- Managed by AWS using **Gateway Load Balancer** behind the scenes
- You define **rules**; no need for third-party appliances
- Integrates with **AWS Firewall Manager** for multi-account management

Capabilities:

- **Rule filtering:** IP, port, protocol, domain names, regex patterns
- **Action options:** Allow, drop, or alert
- **Flow inspection:** Can act as intrusion prevention system
- **Monitoring:** Send logs to **CloudWatch Logs**, **S3**, or **Kinesis Data Firehose**

Example Rules:

- Block SMB outbound traffic
- Allow only traffic to your internal domain or approved repositories
- Detect malicious patterns with regex

Exam Tip:

- Network Firewall = **VPC-level firewall**
- Security Groups & NACLs = more **granular, instance/subnet-level controls**
- WAF = **application layer protection**, mostly HTTP/S
- Shield = **DDoS protection**